



Copiloto de auditoría de inventarios y generación automática de reportes

Reporte Escrito del Reto para AERSA

Por

**Valeria Aguilar Meza
René Abraham Calzadilla Calderón
Victoria González González
Sara Rivera Méndez
Johanna Camila Willis Ruiz**

Para el curso

**Desarrollo de Proyectos de Ingeniería Matemática
MA3001B.501**

1. Resumen

Este proyecto presenta el desarrollo de un copiloto de auditoría para TALOS, sistema de gestión de inventarios utilizado por AERSA en el sector restaurantero. El objetivo fue construir una herramienta que apoyara al auditor en la revisión de inventarios, generación de reportes y consulta de información mediante lenguaje natural. Para ello, primero se realizó un diagnóstico y limpieza de la base `talos_tecmty`, creando vistas SQL que conservan la base original y permiten trabajar con datos más confiables. Después, se implementó una API de generación automática de reportes que consulta las vistas limpias, calcula indicadores, genera gráficas y exporta documentos en PDF. También se desarrolló una API RAG que vectoriza registros de inventario, reportes PDF y reglas de auditoría para recuperar contexto relevante y responder preguntas en español usando Llama 3.2:3b. La solución integra estos componentes mediante una interfaz web y un servidor nginx, permitiendo acceder al chat, reportes, administración y transferencias desde una misma aplicación. Como resultado, el sistema organiza información de inventarios, reduce parte del trabajo manual y funciona como apoyo para la interpretación de diferencias, sin sustituir el criterio final del auditor.

2. Introducción

La auditoría de inventarios en grupos restauranteros implica la revisión periódica de grandes volúmenes de información: diferencias entre stock teórico y físico, variaciones de costo, reajustes, movimientos de compra y requisición, entre otros. La creciente adopción de plataformas digitales en el sector ha incrementado el volumen de datos disponibles, pero no necesariamente la capacidad de analizarlos de forma oportuna [4]. En el caso de AERSA y su plataforma TALOS, un sistema SaaS de gestión de inventarios, compras y ventas orientado a cadenas y grupos de restaurantes, este proceso se presenta en tablas de alta dimensionalidad que los auditores deben analizar de forma manual para realizar reportes, lo que representa una gran carga operativa y una fuente de error humano.

El presente proyecto tiene el objetivo de desarrollar un copiloto conversacional que asista a los auditores en la interpretación de los datos de TALOS, plataforma de gestión avanzada diseñada específicamente para la industria restaurantera por AERSA. Las interfaces en lenguaje natural para bases de datos permiten a usuarios no técnicos formular consultas como lo harían en lenguaje cotidiano, sin necesidad de conocer SQL, reduciendo el tiempo de consulta hasta en un 30 % respecto a plataformas analíticas tradicionales [3]. A diferencia de soluciones de *business intelligence* convencionales, el copiloto propuesto combina esta capacidad con generación de lenguaje natural, permitiendo recibir respuestas contextualizadas y acceder a reportes generados automáticamente.

La solución se fundamenta en la creación de un *Retrieval-Augmented Generation* (RAG) [1], en el que un LLM recibe, además de la pregunta del usuario, fragmentos de información recuperados dinámicamente de una base de datos vectorial. Esta base vectorial se compone de datos de TALOS convertidos a lenguaje natural, reportes de auditoría vectorizados y reglas de auditoría que se integran como fuente de conocimiento del sistema. El modelo de lenguaje empleado es Llama 3.2:3b, una familia de modelos de código abierto desarrollada por Meta [2].

Un componente crítico previo a cualquier análisis fue la preparación de la base de datos `talos_tecmty`, una versión anonimizada de la base productiva de TALOS cedida para fines académicos. Esta base presentaba columnas con nulidad del 100 %, registros en estado incompleto y valores atípicos de magnitud incompatible con el dominio del problema. El proceso de limpieza, diagnóstico y creación de vistas SQL limpias forma parte integral de la metodología descrita en este documento.

El trabajo se estructura de la siguiente manera: la Sección 3 describe la arquitectura del sistema, el proceso de limpieza de datos, la API de reportes, la API RAG, la vectorización y el servidor; la Sección 4 presenta los resultados obtenidos; la Sección 5 discute las implicaciones del diseño; y la Sección 6 sintetiza conclusiones y trabajo futuro.

3. Metodología

3.1. Arquitectura general del sistema

El copiloto se diseñó como una aplicación independiente de la plataforma TALOS. La Figura 1 muestra el flujo general del sistema. El proyecto se organiza alrededor de dos servicios principales. El primero es la API RAG, encargada de indexar datos y responder preguntas en lenguaje natural. El segundo es la API de reportes, encargada de generar reportes de auditoría y manejar transferencias entre almacenes.

La separación en dos APIs permite que cada parte del sistema tenga una responsabilidad clara. La API RAG se concentra en búsqueda semántica y generación de respuestas. La API de reportes se concentra en consultas analíticas, cálculo de indicadores, creación de gráficas y exportación a PDF. Ambas APIs usan la misma base limpia como punto de partida, pero producen salidas distintas: una respuesta conversacional en el chat o un documento formal de auditoría.

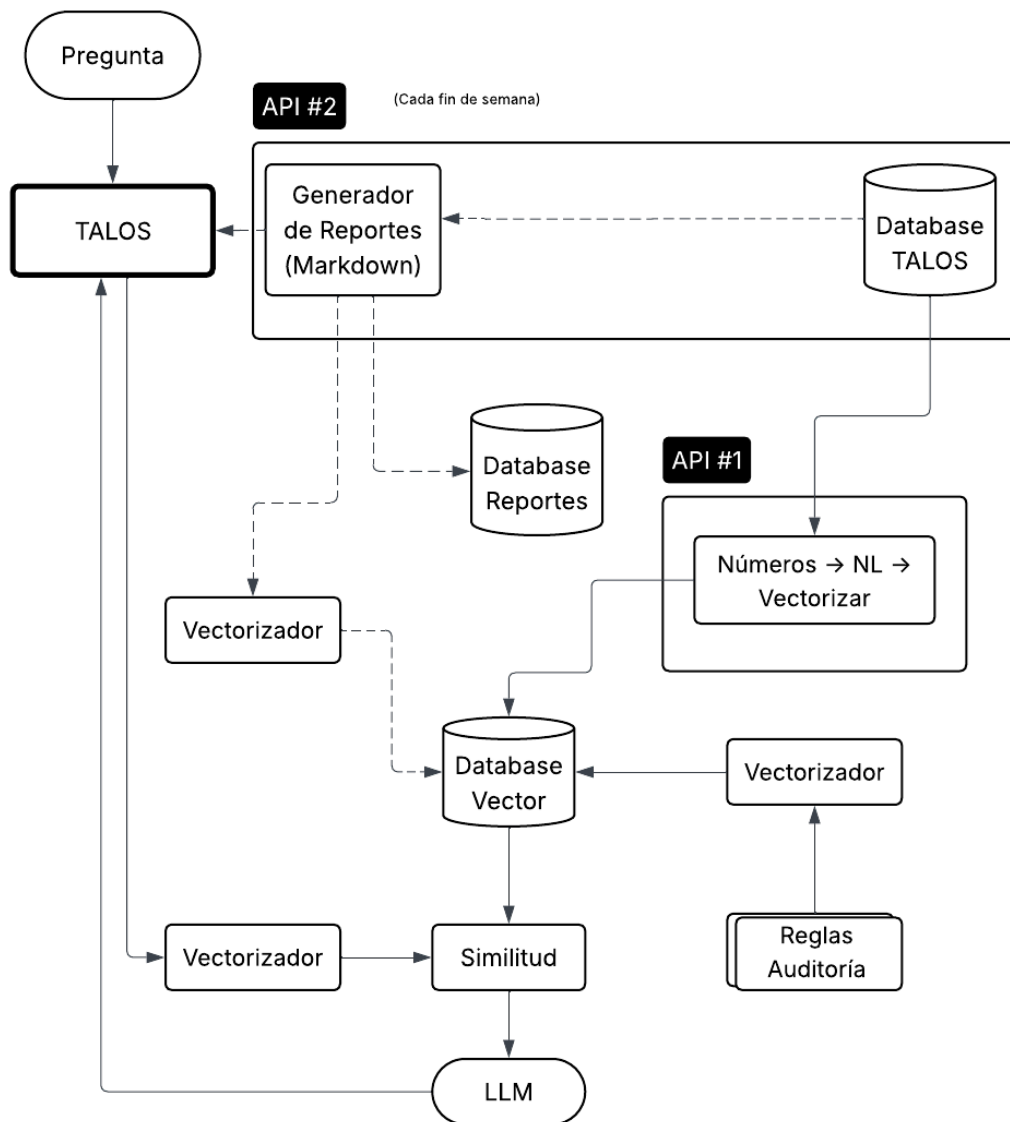


Figura 1: Arquitectura general del copiloto de auditoría para TALOS.

El flujo de consulta funciona de la siguiente manera. El usuario escribe una pregunta en lenguaje natural desde la interfaz web. La API RAG convierte esa pregunta en un embedding y busca documentos similares en Milvus Lite. Los documentos recuperados se concatenan como contexto y se envían a Ollama, donde el modelo Llama 3.2:3b genera una respuesta en español. La similitud se trabaja como similitud vectorial. En los scripts de apoyo se calcula como producto punto entre embeddings normalizados, lo que equivale a similitud coseno; en la API productiva la búsqueda la ejecuta Milvus sobre embeddings normalizados.

En paralelo al flujo de consulta, el sistema mantiene dos procesos de preparación de información:

- **API #1 o API RAG:** indexa datos recientes de inventario y reportes PDF en Milvus Lite. También expone el endpoint de preguntas.

- **API #2 o API de reportes:** genera reportes PDF de auditoría, consulta transferencias confirmadas y permite registrar, detectar, confirmar o rechazar transferencias.

La interfaz web, ubicada en `talos-server/frontend/index.html`, conecta ambos servicios. Incluye una pestaña de chat, una pestaña para consultar y generar reportes PDF, una pestaña de administración para reindexar datos o PDFs, y una pestaña para gestionar transferencias. El archivo `talos-server/nginx.conf` sirve el frontend y redirige solicitudes a la API RAG y a la API de reportes.

El Cuadro 1 resume la organización principal del servidor dentro del repositorio. Esta estructura es importante porque el proyecto no se maneja como un solo archivo, sino como varios módulos conectados entre sí.

Cuadro 1: Estructura principal del servidor en el repositorio.

Ruta	Papel dentro del proyecto
<code>talos-server/api/main.py</code>	API RAG. Recibe preguntas, consulta Milvus Lite y llama al modelo Llama 3.2:3b por medio de Ollama.
<code>talos-server/indexers/</code>	Scripts que preparan datos para Milvus Lite. Incluye el indexador de base de datos y el indexador de reportes PDF.
<code>talos-server/knowledge/</code>	Base de conocimiento con reglas, umbrales y buenas prácticas de auditoría usadas por el RAG.
<code>talos-server/app/main.py</code>	Punto de entrada de la API de reportes. Registra rutas, revisa la conexión a la base de datos y activa el programador semanal.
<code>talos-server/app/routers/</code>	Rutas HTTP de la API de reportes. Incluye reportes y transferencias.
<code>talos-server/app/services/</code>	Lógica interna para procesar datos, generar gráficos, renderizar HTML, crear PDFs y detectar transferencias.
<code>talos-server/app/db/</code>	Conexión asíncrona a la base de datos y consultas SQL usadas por la API de reportes.
<code>talos-server/app/templates/</code>	Plantilla en HTML para la generación de reportes.
<code>talos-server/frontend/index.html</code>	Interfaz web del usuario final. Desde ahí se usa el chat, los reportes, la administración y las transferencias.
<code>talos-server/nginx.conf</code>	Configuración del servidor web que sirve el frontend y redirige solicitudes hacia las APIs.
<code>talos-server/config.py</code>	Configuración compartida de la API RAG: conexión a MySQL, modelo de embeddings, modelo LLM, ruta de Milvus Lite y nombres de colecciones.
<code>talos-server/cron/</code>	Directorio con el script para reindexar periódicamente la información de inventario en la base vectorial.
<code>talos-server/rag/requirements.txt</code>	Dependencias principales para la API RAG y los procesos de vectorización.
<code>talos-server/app/requirements.txt</code>	Dependencias de la API de reportes, incluyendo FastAPI, SQLAlchemy, pandas, matplotlib, Jinja2 y Playwright.
<code>talos-server/docker_readme.md</code>	Documento de apoyo para desplegar el sistema con nginx, APIs, MySQL, Ollama y Milvus Lite.

3.2. Descripción de la base de datos

La base de datos utilizada en este proyecto es **talos_tecmty**, una versión académica de la base productiva de TALOS cedida por AERSA. Contiene datos de 822 empresas cliente y 457 sucursales. El Cuadro 2 describe las tablas disponibles y su contenido.

Cuadro 2: Tablas disponibles en **talos_tecmty**.

Tabla	Descripción
almacen	Catálogo de almacenes por sucursal.
categoria	Catálogo jerárquico de categorías de productos.
inventariomes	Cabecera de cada inventario: fecha, almacén, estatus y totales.
inventariomesdetalle	Detalle producto a producto de cada inventario.
producto	Catálogo propio de productos por empresa.
productotalos	Catálogo homologado de productos entre empresas.
unidadmedida	Catálogo de unidades de medida.

La tabla de mayor volumen es **inventariomesdetalle**, con 11,772,750 registros al momento del análisis. Cada fila representa un producto dentro de un inventario mensual específico e incluye campos de stock inicial, stock teórico, stock físico, diferencia, ingresos y egresos por distintos conceptos, costo promedio e importe físico.

3.3. Diagnóstico y limpieza de la base de datos

La limpieza de datos se realizó antes de cualquier proceso de vectorización o generación de reportes. El objetivo fue conservar la mayor cantidad posible de información útil sin modificar la base de datos original. Para ello se utilizaron vistas SQL, las cuales funcionan como tablas virtuales que almacenan consultas y permiten acceder a datos previamente filtrados o transformados.

Como parte del diagnóstico inicial se ejecutaron consultas sobre cada tabla de la base **talos_tecmty** para comprender la información disponible e identificar posibles problemas de calidad. Se detectaron tres situaciones principales: columnas con alta nulidad, registros inactivos o incompletos y valores atípicos en variables de inventario.

3.3.1. Columnas eliminadas por nulidad total o baja utilidad

En la tabla **producto**, compuesta por 302,325 registros, se identificaron varias columnas que no aportaban información relevante para el análisis debido a su alta proporción de valores nulos o porque ya no tenían uso operativo dentro de la empresa. Las columnas excluidas fueron:

- **producto_descripcion**: nulidad del 100 %.
- **image_path**: nulidad del 99.8 %.
- **created_at**: nulidad del 90.6 %.
- **idimpuesto**: nulidad cercana al 100 % y catalogada como sin uso actual.

- `producto_comentarioreceta`: nulidad del 96.7 %.

De manera similar, en la tabla `almacen`, con 18,387 registros, se eliminó la columna `almacen_fechacreacion`, ya que presentaba nulidad total.

3.3.2. Filtrado de registros inactivos o incompletos

También se aplicaron filtros para excluir registros que no representan información útil para el análisis operativo.

- **Productos dados de baja u ocultos:** se excluyeron 19,264 productos con `producto_baja` = 1 y 6,037 productos con `producto_oculto` = 1.
- **Almacenes inactivos:** de los 18,387 almacenes registrados, 2,835 presentaban `almacen_estatus` = 0, por lo que fueron descartados mediante un JOIN con la tabla `almacen` dentro de las vistas de inventario.
- **Inventarios en edición:** la tabla `inventariomes` contenía 363 registros con estatus `editando`, distribuidos entre 2021 y 2026. Debido a que estos registros corresponden a inventarios abandonados o incompletos, únicamente se conservaron aquellos con estatus `finalizado`, `aplicado` o `terminado`, obteniendo un total de 79,082 inventarios válidos para análisis.

3.3.3. Manejo de valores nulos en el detalle de inventario

En la tabla `inventariomesdetalle` se detectaron 151 valores nulos en el campo de stock físico y 19 valores nulos en el campo de diferencia. Considerando que la tabla contiene 11,772,750 registros, estos casos representan menos del 0.002 % del total.

Eliminar las filas afectadas habría implicado perder información potencialmente valiosa en otros atributos. Por esta razón, se optó por reemplazar los valores nulos mediante la función `IFNULL(..., 0)` dentro de la definición de las vistas SQL.

3.3.4. Detección y marcado de valores atípicos

Durante el análisis de la variable `inventariomesdetalle_stockteorico` se identificaron valores considerablemente alejados de los rangos esperados para la operación de restaurantes:

- Stock teórico máximo: 999,999,999 unidades.
- Stock teórico mínimo: -937,487 unidades.
- 3,521 registros con stock superior a 10,000 unidades.
- 53 registros con stock superior a 100,000 unidades.
- 1,535,714 registros con stock teórico negativo.

- 7,791 registros con stock menor a $-1,000$ unidades.

Asimismo, se identificaron 29 productos que presentaban existencias superiores a 100,000 unidades de manera recurrente en distintos inventarios, lo que sugiere errores acumulados en el historial de movimientos.

En lugar de eliminar estos registros automáticamente, se decidió conservarlos e incorporar un indicador denominado `flag_outlier` dentro de la vista `vw_inventariomesdetalle_limpio`. Esta estrategia permite que los analistas decidan posteriormente si desean excluir dichos registros de ciertos cálculos o revisarlos como posibles errores de captura.

3.3.5. Vistas SQL resultantes

Como resultado del proceso de limpieza se definieron cuatro vistas con la sentencia `CREATE OR REPLACE VIEW`, implementadas en el archivo `sql/view_clean_tables.sql`.

Cuadro 3: Vistas limpias creadas para el análisis.

Vista	Función
<code>vw_producto_limpio</code>	Deja productos activos y visibles, sin columnas de baja utilidad para el análisis.
<code>vw_inventariomes_limpio</code>	Deja inventarios con estatus válido y almacén activo.
<code>vw_inventariomesdetalle_limpio</code>	Prepara el detalle de inventario, reemplaza nulos por cero y agrega banderas de calidad.
<code>vw_categoria_limpia</code>	Catálogo de categorías; no requirió filtros adicionales.

El uso de vistas en lugar de tablas derivadas garantiza que los datos reflejen siempre el estado más reciente de la base original. La aplicación conjunta de todos los filtros resulta en 11,455,135 registros utilizables, equivalentes al 97.3 % del total original. Esto muestra que la mayor parte de la base sí es utilizable, siempre que se trabaje con una capa de limpieza.

El Cuadro 4 resume los cuatro flags de calidad disponibles en `vw_inventariomesdetalle_limpio`.

Cuadro 4: Flags de calidad en `vw_inventariomesdetalle_limpio`.

Flag	Significado cuando vale 1
<code>flag_stockteorico_no_cuadra</code>	El stock teórico guardado no coincide con el cálculo esperado a partir de stock inicial, ingresos, egresos y reajustes.
<code>flag_outlier</code>	Stock, diferencia o reajuste mayor a $\pm 10,000$ unidades.
<code>flag_costopromedio_cero</code>	El costo promedio es cero, por lo que el importe puede no representar correctamente el valor del producto.
<code>flag_fisico_cero_teorico_positivo</code>	El stock físico es cero aunque el stock teórico sea positivo.

3.4. API #2: generación automática de reportes

La API #2 es el componente encargado de transformar los datos de la base `talos_tecmty` en reportes de auditoría. En la versión actual del repositorio se encuentra en `talos-server/app/`. Está construida con FastAPI y Python, se conecta a MySQL mediante SQLAlchemy asíncrono y ejecuta automáticamente cada domingo a las 23:00, hora de México, mediante APScheduler. El reporte resultante se genera como PDF y se almacena localmente.

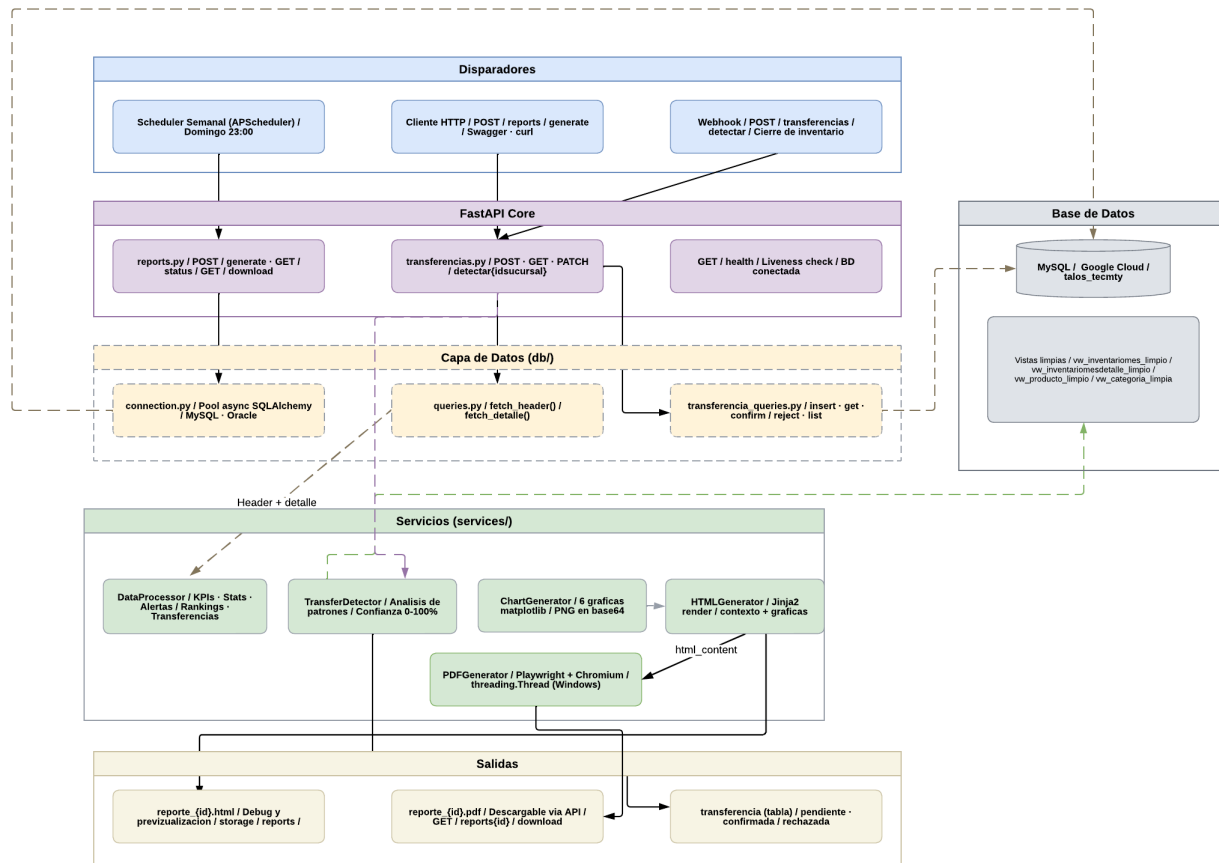


Figura 2: Arquitectura del funcionamiento de la API de generación automática de reportes.

3.4.1. Pipeline de generación

El proceso de generación corre en segundo plano para no bloquear la respuesta HTTP. El flujo completo tiene cinco pasos:

1. **Extracción de datos.** Se consulta la cabecera del inventario, el detalle del inventario y las transferencias confirmadas. Las consultas de la API ya usan las vistas limpias de inventario, detalle, producto y categoría. Para conservar datos adicionales, también hacen JOIN con las tablas originales cuando se necesitan campos que no están en las vistas.
2. **Procesamiento analítico.** El DataProcessor calcula KPIs de inventario, totales de faltantes, sobrantes, balance neto, estadísticos descriptivos, detección de outliers por z-score, clasificación de alertas por prioridad, rankings de productos y ajuste por transferencias confirmadas.

3. **Generación de gráficas.** El `ChartGenerator` crea seis figuras con `matplotlib`, las convierte a PNG y las codifica en base64. Esto permite incrustarlas en HTML y exportarlas a PDF sin depender de JavaScript.
4. **Renderizado HTML.** El `HTMLGenerator` usa Jinja2 para combinar el contexto del `DataProcessor` con las imágenes base64 y renderiza el template `reporte.html`. El HTML generado se guarda además como archivo independiente para facilitar la depuración visual.
5. **Exportación a PDF.** El `PDFGenerator` usa Playwright para abrir Chromium, cargar el HTML renderizado y exportarlo como PDF en formato A4. En Windows el generador corre en un `threading.Thread` nativo aislado del event loop de FastAPI para evitar problemas de compatibilidad.

Cuadro 5: Endpoints principales de la API de reportes.

Endpoint	Uso
GET /health	Revisa el estado del servicio y la conexión con la base de datos.
POST /reports/generate	Inicia la generación de un reporte en segundo plano.
GET /reports/{id}/status	Consulta si el reporte está pendiente o listo.
GET /reports/{id}/download	Descarga el PDF generado.

Cuadro 6: Gráficas generadas por `ChartGenerator` en cada reporte.

Fig.	Tipo	Contenido
1	Pastel	Composición del inventario por categoría: alimentos, bebidas y misceláneos.
2	Barras	Comparación entre faltantes y sobrantes en importe absoluto.
3	Barras agrupadas	Faltantes y sobrantes por categoría.
4	Histograma	Distribución de diferencias en importe, separando faltantes y sobrantes.
5	Barras horizontales	Diez productos con mayor faltante.
6	Mapa de calor	Movimientos por tipo y categoría.

3.4.2. Sistema de detección y registro de transferencias

Un problema recurrente en la auditoría de inventarios es la presencia de transferencias no registradas: cuando un producto se mueve físicamente de un almacén a otro sin capturarse en el sistema, el cierre semanal reporta un faltante en el almacén origen y un sobrante en el almacén destino, inflando artificialmente las pérdidas reales. Para mitigar este problema se implementó un módulo de transferencias en `talos-server/app/routers/transferencias.py` y `talos-server/app/services/transfer_detector.py`.

El módulo contempla tres mecanismos complementarios:

- **Registro manual.** El auditor puede registrar transferencias mediante `POST /transferencias/`; estas quedan automáticamente con estatus `confirmada`.
- **Detección automática.** El componente `TransferDetector` analiza cierres de una sucursal buscando el patrón de un mismo producto con diferencia negativa en un almacén y diferencia positiva en otro almacén. Si las cantidades coinciden dentro de una tolerancia del 15 % y el importe supera un mínimo de \$5, la transferencia se registra como `detectada` con estatus `pendiente`.

- **Flujo de aprobación.** El auditor confirma o rechaza cada transferencia detectada. Una transferencia confirmada se descuenta del faltante bruto para calcular la pérdida real estimada; una transferencia rechazada conserva el faltante como pérdida real.

La sección de transferencias del reporte final muestra el faltante bruto, el importe total de transferencias confirmadas y la pérdida real estimada después del ajuste.

Cuadro 7: Endpoints principales del módulo de transferencias.

Endpoint	Uso
POST /transferencias/	Registra una transferencia manual.
GET /transferencias/{id}	Consulta una transferencia específica.
GET /transferencias/inventario/{id}	Lista transferencias asociadas a un inventario.
GET /transferencias/almacen/{id}	Lista transferencias de un almacén en un rango de fechas, con opción de filtrar por estatus.
PATCH /transferencias/{id}/confirmar	Confirma una transferencia pendiente.
PATCH /transferencias/{id}/rechazar	Rechaza una transferencia pendiente.
POST /transferencias/detectar/{idsucursal}	Detecta posibles transferencias no registradas en una sucursal.

3.4.3. Tecnologías usadas

El Cuadro 8 resume las principales tecnologías empleadas y su rol.

Cuadro 8: Stack tecnológico de la API de reportes.

Tecnología	Uso dentro del módulo
FastAPI	Define endpoints y permite manejar solicitudes HTTP de forma asíncrona.
SQLAlchemy async	Administra la conexión con la base de datos.
pandas	Permite calcular indicadores y transformar datos de inventario antes de generar el reporte.
matplotlib	Genera gráficas estáticas compatibles con PDF.
Jinja2	Separa la plantilla HTML de la lógica de procesamiento.
Playwright	Exporta el HTML final como PDF usando Chromium.
APScheduler	Programa la generación semanal de reportes.

3.5. API #1: servicio RAG y recuperación semántica

La API #1 corresponde al servicio RAG ubicado en `talos-server/api/main.py`. Su función es permitir que el auditor haga preguntas en lenguaje natural sobre la información disponible del proyecto. En lugar de exigir una consulta SQL, el sistema convierte la pregunta en un vector, busca fragmentos cercanos en una base vectorial y usa esos fragmentos como contexto para que el modelo de lenguaje genere una respuesta.

El servicio está construido con FastAPI. Al iniciar, carga una sola vez el modelo de embeddings BAAI/bge-m3 y abre una conexión con Milvus Lite. Esto evita cargar el modelo en cada pregunta, lo cual

haría más lento el sistema. La configuración compartida se encuentra en `talos-server/config.py`; ahí se definen el modelo de lenguaje `11ama3.2:3b`, la URL local de Ollama, la ruta de Milvus Lite y los nombres de las colecciones vectoriales.

El sistema trabaja con tres colecciones vectoriales principales:

- **inventario**: almacena textos generados a partir de registros de inventario de la base SQL.
- **reportes**: almacena fragmentos de texto extraídos de reportes PDF.
- **conocimiento**: almacena reglas, umbrales y buenas prácticas de auditoría para restaurantes y bares.

La consulta puede buscar en una sola fuente de datos o en ambas fuentes operativas. Desde la interfaz, el usuario puede seleccionar inventario, reportes o ambas. Además, la API RAG agrega siempre la colección **conocimiento** al contexto de búsqueda. Esto permite que la respuesta combine datos reales del sistema con criterios de auditoría, por ejemplo umbrales de faltantes, interpretación de sobrantes, señales de riesgo y criterios para revisar transferencias.

3.5.1. Flujo de una pregunta

El endpoint principal es `POST /ask`. La solicitud incluye la pregunta del usuario, el número máximo de fragmentos a recuperar y, de forma opcional, la colección donde se quiere buscar. El flujo interno es el siguiente:

1. La API recibe la pregunta y la convierte en un embedding usando `BAAI/bge-m3`.
2. El embedding se normaliza. Esto significa que el vector se ajusta para que su longitud sea comparable con la de otros vectores.
3. La API revisa qué fuente operativa debe consultar. Si se especifica una colección, busca solo en esa fuente; si no se especifica, consulta **inventario** y **reportes**.
4. Además de la fuente operativa seleccionada, la API agrega la colección **conocimiento** para incluir reglas y criterios de auditoría.
5. Milvus Lite recupera los fragmentos más cercanos a la pregunta. Estos fragmentos se obtienen del campo **texto**, que es donde se guarda la descripción en lenguaje natural.
6. La API concatena los fragmentos recuperados para formar el contexto.
7. El contexto y la pregunta se envían a Ollama mediante una solicitud HTTP local.
8. Ollama ejecuta el modelo `11ama3.2:3b` y devuelve una respuesta en español.

El prompt que recibe el modelo le indica que actúe como asistente experto en auditoría de inventarios para TALOS. También le pide responder en español y decirlo claramente si el contexto recuperado no alcanza para contestar. Esta instrucción ayuda a reducir respuestas inventadas, porque el modelo queda limitado a explicar lo que puede justificar con los fragmentos recuperados.

3.5.2. Búsqueda semántica y similitud

La búsqueda del RAG no depende de que la pregunta tenga exactamente las mismas palabras que los registros. Por ejemplo, una pregunta como “productos con mayor pérdida” puede relacionarse con textos que usan términos como faltante, diferencia negativa o discrepancia de importe. Esto ocurre porque los embeddings representan el significado general del texto, no solamente sus palabras literales.

En el código actual, la API genera embeddings normalizados y le entrega el vector de la pregunta a Milvus Lite. La búsqueda como tal la ejecuta Milvus. En los scripts de apoyo, cuando se comparan embeddings normalizados, el producto punto entre dos vectores equivale a similitud coseno. Por eso, conceptualmente, el sistema trabaja con cercanía semántica tipo coseno, aunque dentro del endpoint `/ask` no se programa manualmente la fórmula de similitud.

3.5.3. Endpoints de indexación

Además de responder preguntas, la API RAG también permite actualizar la base vectorial. Estos endpoints están protegidos con el encabezado `X-API-Key`, ya que modifican el contenido indexado.

- `POST /index/db`: ejecuta el indexador de base de datos. Recibe como parámetros la cantidad de días hacia atrás que se quieren consultar y el límite de registros a insertar.
- `POST /index/pdf`: ejecuta el indexador de PDFs. Recibe la ruta del archivo PDF que se desea convertir a texto y almacenar en Milvus Lite.

La decisión de separar consulta e indexación tiene sentido para este proyecto porque el auditor puede usar el chat sin reindexar datos cada vez. La actualización de índices ocurre solo cuando se agregan nuevos reportes o cuando se quiere refrescar la información reciente de inventarios.

Cuadro 9: Endpoints principales de la API RAG.

Endpoint	Uso
<code>GET /</code>	Verifica que el servicio está activo.
<code>GET /status</code>	Muestra cuántos documentos hay en las colecciones inventario , reportes y conocimiento .
<code>POST /ask</code>	Recibe una pregunta, recupera contexto desde Milvus y genera una respuesta con Ollama.
<code>POST /index/db</code>	Indexa datos recientes de inventario. Requiere encabezado <code>X-API-Key</code> .
<code>POST /index/pdf</code>	Indexa un reporte PDF. Requiere encabezado <code>X-API-Key</code> .

3.6. Vectorización

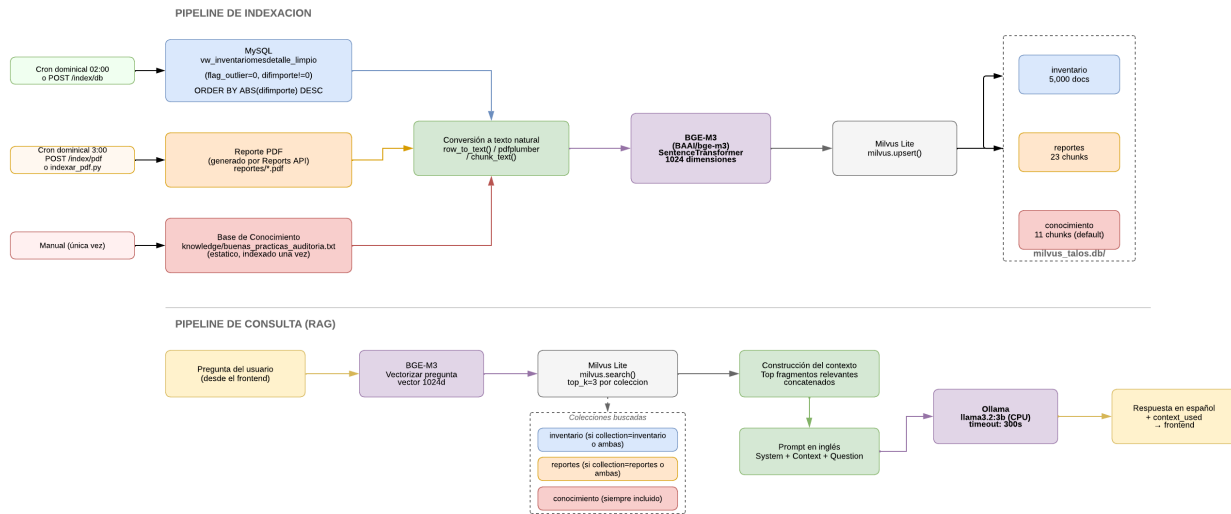


Figura 3: Arquitectura del procesamiento de datos para RAG y vectorización

3.6.1. Vectorización de la base de datos

La vectorización de la base de datos constituye el mecanismo mediante el cual la información estructurada de inventarios es transformada en representaciones vectoriales que pueden ser utilizadas por el sistema de recuperación semántica. Este proceso está implementado en `talos-server/indexers/indexar_db.py`.

El flujo inicia con la extracción de información desde las vistas limpias de inventario, producto y categoría. Durante esta etapa se aplican criterios de filtrado para excluir registros marcados como valores atípicos mediante `flag_outlier` o registros con costo promedio inválido mediante `flag_costopromedio_cero`. Adicionalmente, los datos se restringen a un periodo temporal configurable y se priorizan aquellos registros con mayor impacto económico, utilizando como criterio de ordenamiento el valor absoluto de la discrepancia monetaria detectada.

Una vez obtenidos los registros candidatos, cada fila es transformada en una representación textual descriptiva que integra información de sucursal, almacén, producto, categoría, fecha de inventario, existencias teóricas, existencias físicas, diferencias detectadas y monto económico asociado. Esta transformación permite convertir información tabular en una forma adecuada para su procesamiento mediante modelos de lenguaje.

Posteriormente, las descripciones generadas son procesadas por el modelo de embeddings BGE-M3, el cual produce representaciones vectoriales densas y normalizadas de 1024 dimensiones. Estas representaciones capturan relaciones semánticas entre registros, permitiendo búsquedas basadas en significado en lugar de depender exclusivamente de coincidencias exactas de texto o consultas SQL tradicionales.

Finalmente, los vectores generados y sus correspondientes fragmentos textuales son almacenados en una colección de Milvus Lite llamada `inventario`. Esta base de datos vectorial constituye la capa de recuperación semántica del sistema y permite localizar registros relevantes a partir de consultas expresadas en lenguaje natural.

Cuadro 10: Flujo de vectorización de registros de inventario.

Etapas	Descripción
Extracción	Obtención de registros desde vistas limpias de inventario, producto y categoría.
Filtrado	Eliminación de registros atípicos y datos con costo promedio cero.
Priorización	Ordenamiento por impacto económico de la discrepancia detectada.
Transformación textual	Conversión de cada registro estructurado a una descripción en lenguaje natural.
Vectorización	Generación de embeddings normalizados mediante el modelo BGE-M3.
Persistencia	Almacenamiento de vectores y texto asociado en Milvus Lite.

3.6.2. Vectorización de reportes PDF

Además de la información proveniente de la base de datos operacional, el sistema incorpora conocimiento contenido en reportes de auditoría mediante un proceso de vectorización de documentos PDF. Este componente está implementado en `talos-server/indexers/indexar_pdf.py`.

En la versión actual, el proceso usa `pdfplumber` para extraer texto página por página. Después divide el texto en fragmentos de hasta 800 caracteres con un solapamiento de 100 caracteres. Este solapamiento permite que una idea que queda entre dos fragmentos no se pierda por completo al partir el documento. Los fragmentos demasiado pequeños se descartan para evitar indexar ruido.

Cada fragmento se vectoriza con BGE-M3 y se almacena en Milvus Lite dentro de la colección `reportes`. A diferencia del indexador de base de datos, que trabaja con filas estructuradas, el indexador de PDF trabaja con texto libre extraído del reporte. Esto permite recuperar observaciones, tablas convertidas a texto y explicaciones del documento aunque no estén guardadas como columnas SQL.

Cuadro 11: Flujo de vectorización de reportes PDF.

Etapas	Descripción
Procesamiento del documento	Lectura del PDF con <code>pdfplumber</code> .
Fragmentación	División del texto en chunks de 800 caracteres con 100 caracteres de solapamiento.
Filtrado	Descarte de fragmentos demasiado pequeños.
Vectorización	Generación de embeddings normalizados mediante BGE-M3.
Persistencia	Almacenamiento de vectores y textos asociados en la colección <code>reportes</code> .

La incorporación de reportes PDF vectorizados amplía el contexto disponible para el sistema, ya que permite recuperar información derivada de reportes históricos y no únicamente registros operativos de inventario.

3.7. Base de conocimiento de auditoría

Además de los datos de inventario y los reportes PDF, el servidor incluye una base de conocimiento ubicada en `talos-server/knowledge/buenas_practicas_auditoria.txt`. Este archivo contiene criterios

de auditoría específicos para restaurantes y bares, como umbrales de faltantes, interpretación de sobrantes, uso de z-score, señales de fraude, manejo de transferencias, tasa de revisión y glosario de términos.

En la API RAG esta información se maneja mediante la colección **conocimiento** de Milvus Lite. A diferencia de **inventario** y **reportes**, esta fuente no representa datos de un cierre específico, sino reglas generales para interpretar esos datos. Por esta razón, el endpoint `/ask` la agrega siempre al contexto de búsqueda.

Esto permite que el modelo no solo recupere registros relacionados con la pregunta, sino que también tenga criterios para explicarlos. Por ejemplo, si el usuario pregunta si un faltante es grave, el sistema puede comparar el importe recuperado con umbrales de auditoría definidos en la base de conocimiento.

3.8. Interfaz web y despliegue del servidor

El repositorio incluye una interfaz web en `talos-server/frontend/index.html`. Esta interfaz concentra las funciones principales del sistema en cuatro pestañas:

- **Chat:** permite enviar preguntas a la API RAG y seleccionar si la búsqueda operativa se hará sobre **inventario**, **reportes** o ambas colecciones. La base de conocimiento se agrega automáticamente desde la API.
- **PDF:** permite consultar el estado de un reporte, generar un reporte para un inventario y visualizar o descargar el PDF.
- **Admin:** permite revisar el estado de las APIs, consultar conteos de documentos en Milvus y lanzar procesos de reindexación.
- **Transferencias:** permite cargar transferencias por inventario, confirmar o rechazar transferencias pendientes y ejecutar la detección automática por sucursal.

La interfaz no contiene la lógica pesada del proyecto. Su papel es enviar solicitudes HTTP y mostrar las respuestas al usuario. En el chat, por ejemplo, construye un cuerpo JSON con la pregunta y la colección seleccionada. En la pestaña PDF, envía el identificador del inventario a la API de reportes. En la pestaña Admin, llama a los endpoints de estado e indexación. En Transferencias, consume los endpoints de consulta, confirmación, rechazo y detección automática.

El servidor web se define en `talos-server/nginx.conf`. La ruta raíz sirve el frontend estático. Las solicitudes hacia `/api/rag/` se redirigen al servicio RAG en el puerto 8000, mientras que las solicitudes hacia `/api/reports/` se redirigen a la API de reportes en el puerto 8001. En la versión actual de la interfaz, las constantes del frontend apuntan a estas rutas públicas de nginx, por lo que el navegador no necesita llamar directamente a los puertos internos de cada API.

Cuadro 12: Distribución de responsabilidades en el despliegue.

Componente	Responsabilidad
Frontend estático	Presenta la interfaz y manda solicitudes a las APIs.
Nginx	Sirve el archivo <code>index.html</code> y redirige rutas hacia los servicios internos.
API RAG	Atiende preguntas, consulta Milvus Lite y llama a Ollama.
API de reportes	Genera PDFs, consulta la base SQL y maneja transferencias.
MySQL	Guarda los datos estructurados de TALOS y las vistas limpias usadas por el sistema.
Milvus Lite	Guarda los vectores de inventario, reportes y conocimiento para búsqueda semántica.
Ollama	Ejecuta localmente el modelo <code>llama3.2:3b</code> .

La API de reportes también tiene un proceso de inicio definido en `talos-server/app/main.py`. Cuando el servicio arranca, revisa la conexión a la base de datos, registra los routers de reportes y transferencias, y activa un programador semanal. Este programador usa `APScheduler` y busca inventarios cerrados durante los últimos siete días para generar reportes automáticamente. La fecha, hora y zona horaria se controlan desde la configuración de la aplicación.

Para actualización periódica de la base vectorial, el repositorio incluye `talos-server/cron/cron_db.sh`. Este script activa el entorno de Python y ejecuta el indexador de base de datos, dejando registro en un archivo de log. La indexación de PDFs puede ejecutarse desde la API RAG o desde la pestaña de administración de la interfaz.

En conjunto, el despliegue queda organizado en dos ciclos. El primer ciclo es de uso diario: el auditor entra a la interfaz, pregunta al chat, revisa reportes o valida transferencias. El segundo ciclo es de mantenimiento: se actualiza la base vectorial con datos recientes, se agregan nuevos PDFs cuando existen reportes generados y se generan reportes semanales mediante el programador. Esta separación evita que el uso normal del sistema dependa de ejecutar procesos largos manualmente. A continuación se muestra un diagrama de la arquitectura del servidor en Google Cloud:

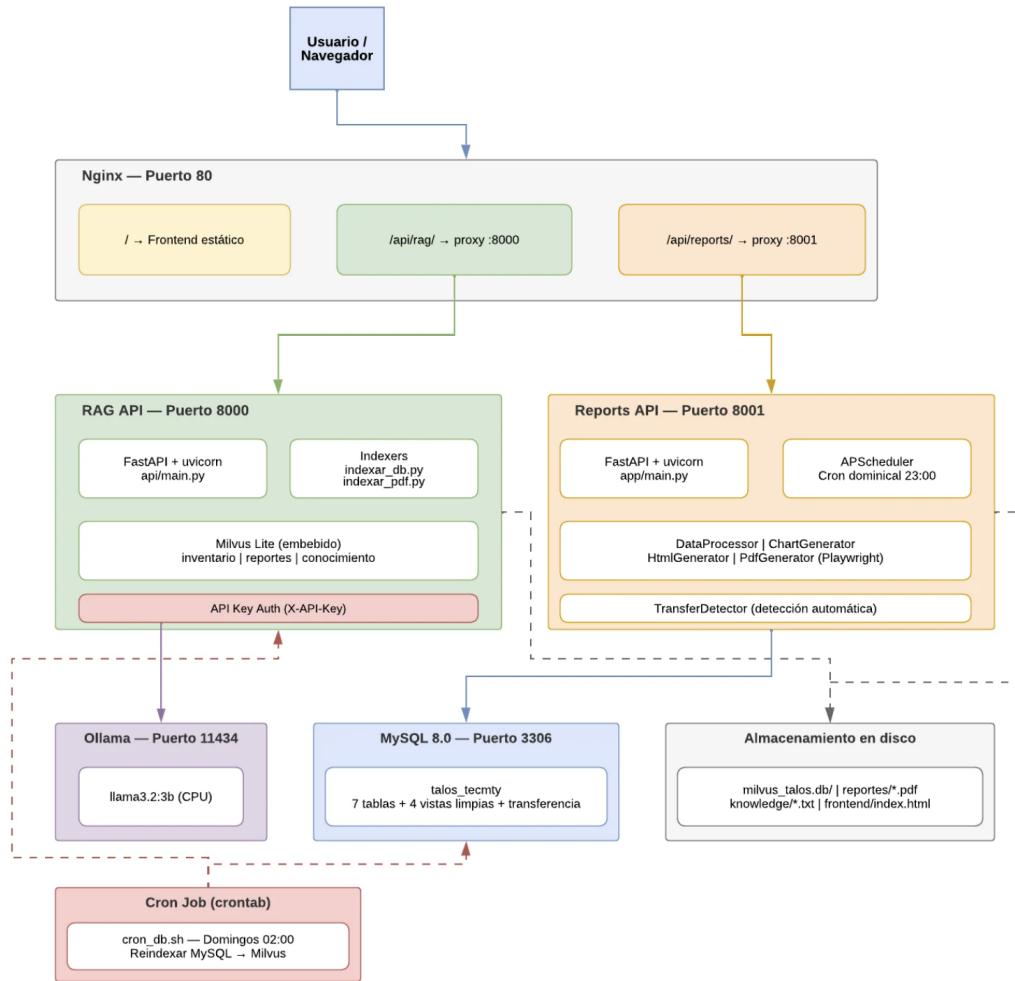


Figura 4: Arquitectura del servidor en Google Cloud

4. Resultados

4.1. Resultados de limpieza y preparación de datos

El primer resultado importante fue la construcción de una capa limpia sobre la base `talos_tecmty`. A partir del diagnóstico inicial se identificaron columnas con alta nulidad, registros inactivos, inventarios incompletos y valores atípicos en variables de inventario. En lugar de modificar directamente la base original, se crearon vistas SQL que permiten trabajar con datos filtrados y campos preparados para análisis.

La limpieza permitió conservar 11,455,135 registros utilizables de la tabla de detalle de inventario, equivalentes al 97.3 % del total original. Este resultado es relevante porque muestra que la mayor parte de la información puede aprovecharse si se trabaja con una capa de validación previa. Además, los registros con posibles problemas no se eliminan sin explicación, sino que se identifican con flags de calidad.

Cuadro 13: Resultados principales del proceso de limpieza.

Resultado	Descripción
Vistas limpias	Se definieron vistas para productos, inventarios, detalle de inventario y categorías.
Conservación de datos	Se mantuvo el 97.3 % de los registros originales del detalle de inventario como información utilizable.
Flags de calidad	Se agregaron indicadores para detectar datos atípicos, costos promedio en cero, stock teórico inconsistente y productos con físico cero aunque deberían tener existencia.
Base original intacta	La limpieza se implementó mediante vistas, por lo que la información original no se modifica.

4.2. Resultados de vectorización y recuperación

El sistema logró transformar información estructurada y no estructurada en datos consultables mediante búsqueda semántica. Para la base de datos, el indexador convierte registros de inventario en descripciones en lenguaje natural que incluyen sucursal, almacén, producto, categoría, fecha, stock teórico, stock físico, diferencia y monto económico. Estas descripciones se vectorizan con BGE-M3 y se almacenan en la colección **inventario**.

Para los reportes PDF, el sistema extrae texto con **pdfplumber**, lo divide en fragmentos de tamaño controlado y almacena sus embeddings en la colección **reportes**. Además, se agregó una base de conocimiento de auditoría en la colección **conocimiento**, la cual aporta reglas generales y criterios de interpretación. Con esto, el RAG puede combinar registros operativos, reportes históricos y criterios de auditoría dentro del mismo flujo de respuesta.

Cuadro 14: Fuentes de información disponibles para el RAG.

Colección	Información que aporta
inventario	Registros recientes de inventario convertidos a texto y priorizados por impacto económico.
reportes	Fragmentos de reportes PDF que contienen observaciones, tablas y explicaciones de auditoría.
conocimiento	Reglas, umbrales, señales de alerta y buenas prácticas para interpretar resultados de inventario.

Como resultado, la API RAG ya cuenta con endpoints para consultar el estado de las colecciones, responder preguntas e iniciar procesos de indexación. El endpoint **/ask** recupera contexto desde Milvus Lite y lo envía a Llama 3.2:3b por medio de Ollama, permitiendo que el auditor reciba una respuesta en español basada en fragmentos relacionados con su pregunta.

4.3. Resultados de generación de reportes

La API de reportes permite generar un documento PDF a partir de un inventario específico. El proceso incluye la consulta de datos desde las vistas limpias, el cálculo de indicadores, la generación de gráficas, el

renderizado de una plantilla HTML y la exportación final a PDF. El reporte no se limita a mostrar tablas, sino que organiza los datos en secciones de análisis, rankings, alertas, movimientos y transferencias.

Un resultado importante de este módulo es que la generación se ejecuta en segundo plano. Esto evita que la solicitud HTTP quede bloqueada mientras se procesa el reporte. Además, el servicio cuenta con endpoints para iniciar la generación, consultar el estado del reporte y descargar el PDF cuando está listo.

El módulo de transferencias complementa el reporte porque permite distinguir entre faltantes reales y movimientos físicos no registrados. Las transferencias manuales se registran como confirmadas, mientras que las transferencias detectadas automáticamente quedan pendientes para que el auditor las revise. Cuando una transferencia se confirma, su importe se descuenta del faltante bruto para estimar la pérdida real.

4.4. Resultados de integración del servidor

El proyecto también cuenta con una integración funcional entre frontend, API RAG, API de reportes y servidor web. La interfaz concentra el chat, la consulta de PDFs, las funciones administrativas y la gestión de transferencias. Nginx sirve el frontend y redirige las solicitudes hacia la API correspondiente, lo que permite usar el sistema desde una sola entrada aunque internamente existan dos servicios principales.

Cuadro 15: Resumen de componentes implementados.

Componente	Resultado obtenido
Limpieza de datos	Vistas SQL limpias y flags de calidad para preparar la base antes del análisis.
Vectorización de base	Registros de inventario transformados en texto y almacenados como embeddings en Milvus Lite.
Vectorización de PDFs	Reportes procesados por fragmentos e incorporados a la colección reportes .
Base de conocimiento	Reglas de auditoría agregadas como contexto permanente para el RAG.
API RAG	Servicio capaz de recuperar contexto y generar respuestas en español con Llama 3.2:3b.
API de reportes	Servicio capaz de generar PDFs de auditoría a partir de inventarios específicos.
Transferencias	Registro, detección, confirmación y rechazo de transferencias entre almacenes.
Frontend y nginx	Interfaz web conectada a ambas APIs mediante rutas públicas del servidor.

5. Discusión

Los resultados muestran que el proyecto avanzó en dos necesidades principales del proceso de auditoría: preparar datos confiables y facilitar su consulta. La limpieza de datos fue un paso necesario porque el sistema depende de información histórica con registros incompletos, productos inactivos y valores atípicos. Sin esta capa previa, tanto los reportes como el RAG podrían dar respuestas basadas en datos que no representan correctamente la operación.

El uso de vistas SQL fue una decisión adecuada para este contexto. Las vistas permiten mantener la base original sin cambios y, al mismo tiempo, ofrecen una versión más útil para análisis. Esto es importante porque la base viene de un sistema operativo real, donde no siempre conviene modificar tablas originales solo para fines de análisis académico. Además, las vistas facilitan que los componentes posteriores trabajen con la misma lógica de limpieza.

La vectorización de la base de datos permite que la información tabular se consulte de una forma más flexible. En vez de depender únicamente de consultas SQL exactas, el auditor puede hacer preguntas en lenguaje natural. Sin embargo, esta decisión también implica una limitación: el resultado depende de cómo se redactan los textos generados para cada registro. Si una descripción omite información importante, esa información tampoco estará disponible para la búsqueda semántica. Por esta razón, la transformación de filas a texto es una parte central del diseño del RAG.

La incorporación de reportes PDF y de una base de conocimiento mejora el contexto disponible para el modelo. Los reportes aportan información ya interpretada, mientras que la base de conocimiento aporta criterios generales de auditoría. Esto ayuda a que las respuestas no solo mencionen datos, sino que también expliquen por qué un resultado puede ser relevante. Aun así, el sistema debe tratarse como una herramienta de apoyo para el auditor, no como sustituto de la revisión humana. En particular, las transferencias detectadas automáticamente requieren confirmación porque una coincidencia entre faltante y sobrante puede sugerir un movimiento no registrado, pero no lo prueba por sí sola.

La API de reportes resuelve otra parte importante del problema: reducir el trabajo manual necesario para preparar documentos de auditoría. El pipeline automatiza consultas, cálculos, gráficas y exportación a PDF. Esto puede ahorrar tiempo al equipo auditor y también ayuda a estandarizar la forma en que se presentan los resultados. La principal limitación pendiente es validar los reportes con más casos reales, revisar que las gráficas sean claras para los usuarios finales y ajustar el contenido según las necesidades de AERSA.

En cuanto al servidor, la separación entre API RAG y API de reportes facilita el mantenimiento porque cada servicio tiene una responsabilidad distinta. Nginx permite que ambas APIs se usen desde una sola interfaz, lo cual simplifica la experiencia del usuario. Para un entorno productivo, todavía sería recomendable mover credenciales y llaves de API a variables de entorno, revisar la configuración de CORS y definir una estrategia más formal para respaldar los archivos generados y la base vectorial.

En general, el proyecto demuestra que es viable construir un copiloto de auditoría que combine limpieza de datos, generación automática de reportes y recuperación semántica. La parte más importante para seguir avanzando será evaluar la calidad de las respuestas del RAG, medir el tiempo de generación de reportes, validar los resultados con usuarios del área de auditoría y ajustar los criterios de la base de conocimiento conforme se obtenga retroalimentación.

6. Conclusión

El proyecto cumplió con el objetivo de construir una solución de apoyo para la auditoría de inventarios en TALOS. A partir de la limpieza de datos, la generación automática de reportes y el uso de recuperación semántica, el sistema permite consultar información de inventarios de forma más clara y reducir parte del trabajo manual asociado al análisis.

La creación de vistas limpias permitió trabajar con datos más confiables sin modificar la base original. Sobre esa base se desarrollaron dos componentes principales: una API de reportes, que calcula indicadores y genera documentos PDF, y una API RAG, que combina registros de inventario, reportes y reglas de auditoría para responder preguntas en lenguaje natural.

En conjunto, el sistema funciona como un copiloto para el auditor: organiza información, recupera contexto relevante y ayuda a interpretar diferencias de inventario. La decisión final sigue dependiendo del criterio humano, especialmente en transferencias detectadas automáticamente o posibles señales de riesgo.

Como trabajo futuro, sería conveniente evaluar las respuestas con más casos reales, ajustar los umbrales de auditoría con retroalimentación de AERSA y reforzar la configuración de despliegue, respaldo de archivos y control de acceso.

Referencias

- [1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel y Douwe Kiela. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arXiv:2005.11401, 2020. Disponible en: <https://arxiv.org/abs/2005.11401>
- [2] Aaron Grattafiori et al. *The Llama 3 Herd of Models*. arXiv:2407.21783, 2024. Disponible en: <https://arxiv.org/abs/2407.21783>
- [3] Panos Ipeirotis y Haotian Zheng. *Natural Language Interfaces for Databases: What Do Users Think?* arXiv:2511.14718, 2025. Disponible en: <https://arxiv.org/abs/2511.14718>
- [4] Mohamad Khalil, A. Stephen McGough, Zoya Pourmirza, Mehdi Pazhoohesh y Sara Walker. *Machine Learning, Deep Learning and Statistical Analysis for Forecasting Building Energy Consumption: A Systematic Review*. Engineering Applications of Artificial Intelligence, vol. 115, p. 105287, 2022. Disponible en: <https://www.sciencedirect.com/science/article/pii/S0952197622003372>
- [5] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian y Zheng Liu. *M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. arXiv:2402.03216, 2024. Disponible en: <https://arxiv.org/abs/2402.03216>